

10

10주차: 이미지 생성형 모델 GAN & Diffusion

Generative Adversarial Nets

arxiv.org

<https://arxiv.org/pdf/1406.2661>

Introduction

딥러닝의 목표) 이미지, 음성, 자연어 같은 데이터의 확률 분포를 잘 표현하는 모델을 만드는 것

문제제기

지금까지는 입력 → 정답(label)으로 매핑하는 discriminative model이 성공적이었지만 Generative model은 확률 계산이 어렵고 학습이 복잡해서 상대적으로 성과가 적었다.

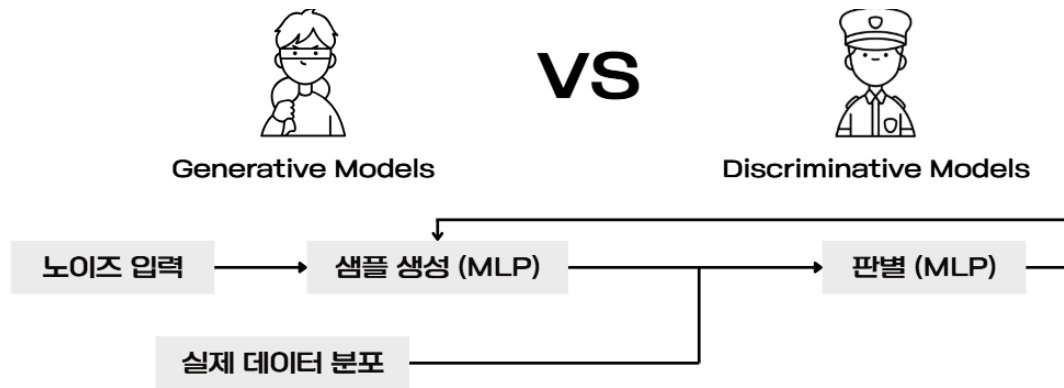
아이디어 제시

적대적 구조 (adversarial framework)를 제안

- 생성자(G): 가짜 데이터를 만들어냄
- 판별자(D): 진짜 vs 가짜 구별

⇒ 둘이 서로 경쟁하면서 성능이 향상된다.

예를 들어 G는 위조지폐 제작자, D는 경찰이라고 비유했을 때 경찰이 판별할 수 없도록 위조지폐가 점점 정교해지는 과정인 것이다.



Adversarial nets

Adversarial modeling framework란?

- 여러개의 딥러닝 모델들을 학습시킬 때 가장 간단하게 적용할 수 있는 방식
- generative 모델과 discriminative 모델 정의
 - 입력 데이터 x 에 대한 generator의 분포 p_g 를 학습하기 전에 input noise 변수 $p_z(x)$ 를 정의하고 $p_z(x)$ 를 data space에 매핑하는 $G(z; \theta_g)$ 를 정의한다.
*여기서 $G = \theta_g$ 로 이루어진 multilayer perceptron
 - 다른 multilayer perceptron $D(x; \theta_d)$ 도 정의합니다. D 는 입력 데이터 x 가 p_g 가 아니고 진짜 데이터 분포일 확률을 single scalar로 나타내는 함수이다.
- 학습 목표

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- D : 학습 데이터셋과 G 에서 만든 데이터에 correct label을 할당할 확률을 최대값으로 만드는 것
- G : D 가 correct label을 제대로 할당할 수 없을 정도로 학습 데이터셋과 유사한 데이터를 만드는 것.
(논문에서는 이를 " $\log(1-D(G(z)))$ 를 최소값으로 만들려는 것($\log 1 = 0$)"이라고 함)
- 학습 과정

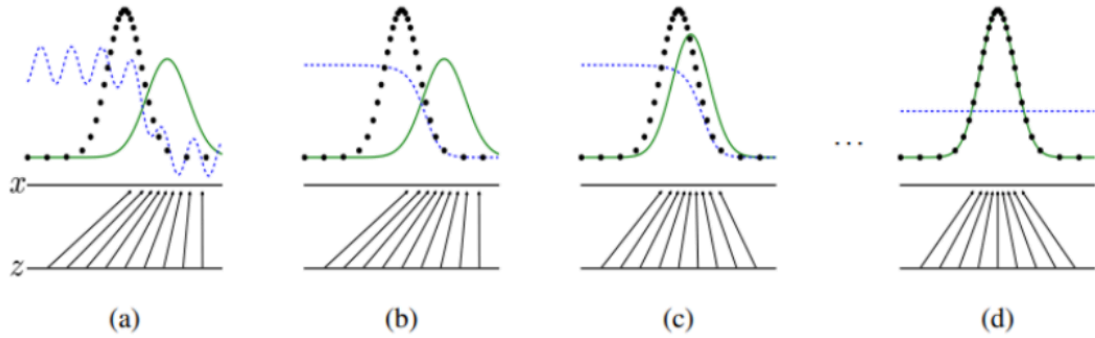


Figure 1

- Noise로부터 G 를 통해 가짜 데이터를 만들고 이를 D 에 Input으로 넣습니다.
- 일반적인 Neural Network는 Input과 Hidden Layer를 거쳐 Output을 계산하는 것까지를 Feed Forward라 하는데, G 는 G 의 아웃풋을 다시 D 의 Input으로 넣어 Output까지 계산한다. **G 의 목적이 D 를 속이는 것이고 D 의 에러를 통해 G 의 Back Propagation을 계산하기 때문에 G 의 Feed Forward는 D 까지 거치게 되는 것이다.**
- G 의 입장에서 상기시켜보면 G 의 목적은 D 를 속이기 위해서다. G 는 D 를 속이는 방향으로 G 의 Weight를 업데이트 해야 한다. 즉, D 의 입장에서 G 가 만들어낸 가짜 데이터를 진짜로 판단하도록 G 의 웨이트를 업데이트 하는 것 이다.

⇒ 이 과정을 위해 D 의 에러에서부터 G 까지 weight를 역전파 시킨다. 이때 D 의 웨이트는 업데이트하지 않는다. D 는 단지 G 의 웨이트를 구하기 위해 에러를 전파시키기만 할 뿐, 웨이트는 업데이트 하지 않습니다. (= Weight Freezing)

이 과정을 무한히 반복하면 진짜 데이터와 가짜 데이터를 구분할 확률이 0.5에 수렴한다

Theoretical Results

G = 확률 분포 p_g 를 $z \sim p_z$ 일 때 얻은 표본 $G(z)$ 의 분포로 암묵적으로 정의한다. 따라서 저자는 충분한 양과 훈련 시간이 주어진다면 Algorithm 1이 p_{data} 의 좋은 추정치로 수렴되기를 바란다. 이 섹션의 결과는 non-parametric하게 수행된다. 예를 들어 확률 밀도 함수의 공간에서의 수렴하는걸 학습함으로 써 무한 용량을 가진 모델을 표현하겠다고 말했다.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Algorithm1_적대신경망의 훈련을 위한 미니배치 확률적 경사 하강법, 하이퍼파라미터 k 는 구별모델에 적용하기 위한 스텝을 나타낸다. 저자는 실험에서 $k = 1$ 을 사용하였다.

1. noise prior $p_g(z)$ 로부터 noise 시킨 m 개의 미니배치 $\{z(1), \dots, z(m)\}$ 샘플을 만든다.
2. 데이터 생성분포 $p_{\text{data}}(x)$ 로부터 미니배치 m 개의 샘플 $\{x(1), \dots, x(m)\}$ 을 평가한다.
3. 확률적 경사 상승법을 이용하여 D 를 업데이트한다.

k 번의 스텝 이후

$k-1$. noise prior $p_g(z)$ 로부터 noise 시킨 m 개의 미니배치 $\{z(1), \dots, z(m)\}$ 샘플을 만든다.

$k-2$. 확률적 경사 하강법을 이용하여 G 를 업데이트한다.

이 실험은 momentum을 이용한 옵티마이저를 사용했다.

(1) Global Optimality of $p_g = p_{\text{data}}$

minmax game이 $p_g = p_{\text{data}}$ 에 대한 global optimum임을 보여줄 것이다.

우선 주어진 G 에 대해 최적의 D 가 있다고 가정.

Proposition1

$$D_G^*(\mathbf{x}) = \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})}$$

G가 고정되어 있을 때 최적의 D

$$\begin{aligned} V(G, D) &= \int_{\mathbf{x}} p_{data}(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathbf{z}} p_{\mathbf{z}}(\mathbf{z}) \log(1 - D(g(\mathbf{z}))) d\mathbf{z} \\ &= \int_{\mathbf{x}} p_{data}(\mathbf{x}) \log(D(\mathbf{x})) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) d\mathbf{x} \end{aligned}$$

proof 1) 주어진 어떤 G에 대해 D의 훈련법은 V(G,D)의 값을 최대화 시키는 것이다.

이 식이 최대값을 갖기 위해 만족해야할 조건을 찾아보자면, (0,0)이 아닌 $(a, b) \in \mathbb{R}^2$ 가 있을 때 $y \rightarrow a \log(y) + b \log(1 - y)$ 가 최대값을 가지려면 어떻게 하면 되는지 알게되면 $V(G, D)$ 가 최대값을 가질 조건을 알 수 있다. 미분한 값이 0이 되는 시점의 y를 알면 된다.

미분을 하면 $a/y - b/(1-y)$ 가 되고 $a/y - b/(1-y) = 0$ 이라는 식을 세우고 y에 대해 정리를 하면 $y = a/(a + b)$ 가 된다. 이를 $V(G, D)$ 의 값을 보며 a, b를 바꿔보면 $a = p_{data}(x), b = p_g(x)$ 가 된다.

그럼 앞선 Proposition에서 세운 최적의 D가 나오게 되고 최적의 D는 올바른 식이라는 것을 알 수 있다.

D를 학습시키는 목적은 조건부 확률 $P(Y=y|x)$ 의 log-확률(log-likelihood)을 최대화하기 위함으로 해석될 수 있다. 여기서 Y는 입력 데이터 x가 진짜 데이터에서 온건지 아니면 모방해서 만든 데이터에서 온건지 나타낸다.

그러므로 앞선 minmax게임을 다음과 같은 global minimum of the virtual training criterion $C(G)$ 로 재구성 할 수 있다.

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - D_G^*(G(\mathbf{z})))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] \end{aligned}$$

Theorem 1. 가상 훈련 기준 $C(G)$ 의 전역 최소값은 $p_g = p_{data}$ 인 경우에만 달성된다. 이 시점에서 $C(G)$ 는 $-\log(4)$ 라는 global minimum 값을 달성합니다.

$p_g = p_{data}$ 에 대해 $D_G(x) = 0.5$ 이다. 그렇다면

$$C(G) = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4.$$

$$\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [-\log 2] + \mathbb{E}_{\mathbf{x} \sim p_g} [-\log 2] = -\log 4$$

이 식을 $C(G) = V(D_G, G)$ 에서 빼면 다음과 같은 값을 얻을 수 있다.

$$C(G) = -\log(4) + KL \left(p_{\text{data}} \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right) + KL \left(p_g \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right)$$

KL은 **Kullback-Leibler** 발산이다.

우리는 이전 표현에서 모델의 분포와 데이터 생성과정에서 Jensen-Shannon 발산을 발견했다.

$$C(G) = -\log(4) + 2 \cdot JSD(p_{\text{data}} \| p_g)$$

두 분포 사이의 Jensen-Shannon 발산은 항상 음이 아니고 그것들이 같을 때만 0이기 때문에, 우리는 $C^* = -\log(4)$ 가 $C(G)$ 의 global minimum이며, $-\log(4)$ 를 얻을 수 있는 유일한 조건은 $p_g = p_{\text{data}}$ 이라는 걸 증명 했다.

즉, 생성모델이 완벽하게 실제 training data 분포를 완벽히 복제해야 virtual training criterion $C(G)$ 의 global minimum을 얻을 수 있는 것이고 이는 생성모델이 데이터를 완벽하게 생성했다고 볼 수 있다.

(2) Convergence of Algorithm 1

Algorithm 1의 value function $V(G, D)$ 를 최적화하여 원하는 결과를 얻는다는 것을 보여줄 것이다.

Proposition 2

G 와 D 가 충분한 능력을 가지고 있고 Algorithm 1의 각 단계에서 D 는 최적의 G 에 도달할 수 있으며 p_g 는 criterion을 향상 시키는 방향으로 업데이트 되어 p_{data} 로 수렴한다.

위에서 쓰인 기준과 같이 $V(G, D) = U(p_g, D)$ 를 p_g 의 함수로 간주한다.

$U(p_g, D)$ 는 p_g 에 있는 볼록한 부분(convex)이라는 걸 눈여겨 봐야한다.

볼록 함수의 상한을 하위미분한 것은 최대값을 얻을 수 있는 점에서 함수를 미분한게 포함되어 있는데, 수학적으로는 ' $f(x) = \sup_{\alpha \in A} f_{\alpha}(x)$ 와 $f_{\alpha}(x)$ 가 모든 α 에 대해 x 에서 convex 일 때, $\beta = \operatorname{argsup}_{\alpha \in A} f_{\alpha}(x)$ 인 조건 아래 $\partial f_{\beta}(x) \subset \partial f$ 이다'로 표현할 수 있다.

⇒ corresponding한 G 가 있을 때 최적의 D 에서의 p_g 를 만드는데 사용될 gradient를 계산하는게 $U(p_g, D)$ 에 들어있다

$\sup_D U(p_g, D)$ 는 $C(G)$ 를 증명할 때 증명되었듯 균일한 global optima를 가진 p_g 에 있는 convex이므로 p_g 를 조금씩, 그러나 충분할 정도로 업데이트 하다보면 p_g 는 p_x 에 수렴하게 된다.

⇒ 이는 해당 G 가 주어진 최적 D 에서 p_g 에 대한 경사 하강법을 계산하는 것과 같다.

이론적으로는 이렇게 완벽한 D , G 를 만들 수 있지만 실제로 만들려고 할 때는 그렇지 않습니다.

adversarial nets은 $G(z; \theta_g)$ 를 통해 제한된 값의 집합인 p_g 분포를 나타내게 되므로, p_g 자체보다는 G 의 parameter인 θ_g 를 최적화 했다고 한다. p_g 를 수렴시킨다는 걸로 증명한게 적용되지 않았다 볼 수 있다. 그러나, 최적화된 θ_g 를 가진 G 는 실전에서 뛰어난 성능을 자랑한다. GAN이 이론적으로 보장이 없음에도 불구하고 사용할 수 있는 합리적인 모델임을 보여준다.

Experiments

- MNIST, TFD, CIFAR-10를 포함한 데이터들로 학습
- generator nets (G)는 rectifier linear activations와 sigmoid activations를 혼합
- discriminator nets (D)는 maxout activations를 사용
- D 를 학습시킬때 dropout을 적용
- G 의 제일 하위 layer에만 input으로 noise를 사용

G 를 통해 생성된 sample들에 Gaussian Parzen window를 fitting하고 이 분포 하의 log-likelihood를 reporting함으로써 p_g 하에서 test set data의 확률을 측정하였다.

Gaussian의 σ 는 validation set에 대한 cross validation을 통해 얻었다.

이 방식은 다양한 생성 모델에서 likelihood가 계산할 수 없을 경우에 사용된다. likelihood를 추정하는 이 방식은 분산이 크고 고차원 공간에서 잘 수행되지는 않지만 저자들의 지식에서는 가능한 최선의 방법이라고 한다. 그리고 likelihood를 직접 추정하지 못하는 생성모델의 발전은 이러한 모델들을 어떻게 평가할지에 대한 앞으로의 research를 motivate한다. G 로 생성한 sample들은 더 좋은 생성모델들에 최소한 경쟁력을 지닌다고 저자들은 믿으며, adversarial framework의 잠재력을 강조한다고 한다.

Advantages and disadvantages

장점

- Markov Chain이 필요없어서 기존 생성 모델과 달리 샘플링 과정이 매우 단순하다.
- 복잡한 확률 계산없이 gradient 기반 학습 가능하다.
- 모델에 다양한 함수가 포함될 수 있다.

단점

- 명시적인 확률 분포가 없다.
 $p_g(x)$ 를 직접적으로 표현하지 않아서 likelihood 계산이 불가능하다. 그래서 다른 생성 모델처럼 확률 기반 평가가 어렵다.
- 학습 안정성이 매우 어렵다.
G와 D의 균형이 중요하고 둘 중 하나가 너무 강해지면 학습이 붕괴된다.
- G가 다양한 데이터를 생성하지 못하고 같은 출력만 반복 생성한다.
예를 들어 5000원을 잘 만들면 1000원, 10000원 안만들고 5000원만 계속 만드는 거임.

Conclusions and future work

- G와 D에 c 를 추가함으로써 conditional generative model $p(x|c)$ 를 얻을 수 있다.
- auxiliary network를 x 가 주어졌을 때 z 를 예측하도록 학습시킴으로써 Learned approximate inference가 수행될 수 있다. wake-sleep 알고리즘에 의해 inference net이 학습되어지는 것과 유사하지만, generator net이 학습이 완료되고 난 후 고정된 generator net를 위해 inference net이 학습 되어질 수 있다는 이점을 가지고 있다.
- parameters를 공유하는 conditionals model를 학습함으로써 다른 conditionals models을 근사적으로 모델링할 수 있다. 특히 MP-DBM의 stochastic extension의 구현에 대부분의 네트워크를 사용할 수 있다.
- Semi-supervised learning : discriminator 혹은 inference net으로부터의 feature들이 labeled data가 제한될 때 classifier들의 성능을 향상시킬 수 있다.
- Efficiency improvements : G와 D를 조정하는 더 좋은 방식들을 고안하거나 학습동안 sample z 에 대한 더 좋은 분포를 결정함으로써 학습은 가속화될 수 있다.

Denoising Diffusion Probabilistic Models

Introduction

Diffusion model이란?

- parameterized Markov chain을 학습시켜 유한 시간 후에 원하는 데이터에 맞는 샘플을 만드는 모델.
- Forward process에서는 Markov chain이 점진적으로 noise를 추가하여 최종적으로 가우시안 noise로 만든다.
- 반대로 reverse process는 가우시안 noise로부터 점진적으로 noise를 제거하여 최종적으로 원하는 데이터에 맞는 샘플을 만든다.
- Diffusion이 작은 양의 가우시안 noise로 구성되어 있기 때문에 샘플링 chain을 조건부 가우시안으로 설정하는 것으로 충분하고, 간단한 신경망으로 parameterize할 수 있다.

⇒ 이전까지 diffusion 모델이 고품질의 샘플을 생성할 수 있다고 입증 X

아이디어 제시 ⇒ DDPM

- 고품질 샘플을 만들 수 있을 뿐만 아니라 다른 생성 모델보다 더 우수한 결과를 보였다.
- diffusion model의 특정 parameterization이 학습 중 여러 noise 레벨에서의 denoising score matching과 비슷하며, 샘플링 중 Langevin dynamics 문제를 푸는 것과 동등하다는 것을 보였다.

Background

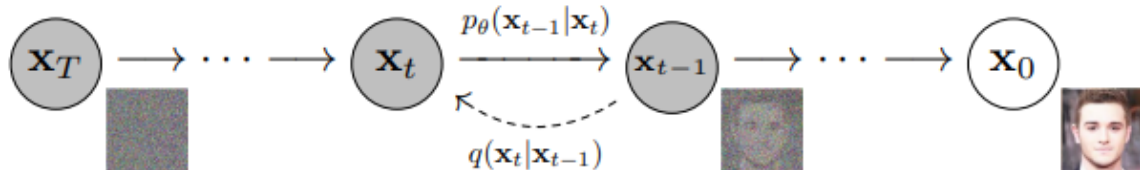


Figure 2: The directed graphical model considered in this work.

Reverse Process

Forward Process를 통해 이미지를 완전한 노이즈로 만들었으면 우리는 그 반대 방향인 노이즈를 지우는 reverse process를 알아내야한다. 우리가 알고싶은 것은 $q(x_{t-1}|x_t)$ 이다.

하지만 이를 알기 힘들므로 approximation인 $p_\theta(x_{t-1}|x_t)$ 를 구하는 것이 목적이다.

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t),$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)), (1)$$

Reverse process또한 forward process와 마찬가지로 Markov Chain이다. 완전한 노이즈 $p(x_T) = N(x_T; 0, I)$ 에서 시작한다. β_t 가 작으면 $q(x_t|x_{t-1})$ 와 $q(x_{t-1}|x_t)$ 가 같은 형태를 띤다고 증명이 됐기 때문에 p_θ 가 가우시안분포라고 할 수 있다.

Forward Process

forward process는 이미지에 노이즈를 주입해주는 process이다.

이미지 x_0 에 가우시안 노이즈를 점점 추가해줌으로써 최종적으로 완전한 가우시안 노이즈 $x_T \sim N(x_T; 0, I)$ 를 얻게 된다.

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}),$$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t I),$$

여기서 $t-1$ 시점에서 노이즈를 추가해서 t 시점의 분포 $q(x_t|x_{t-1})$ 를 얻기에 Markov Chain이다. 또한 아래 식으로 두는 이유는 x_t 의 분산이 너무 커지는 것을 방지하기 위해서이다.

Negative log likelihood

$$\mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

모델이 이미지를 잘 복원하도록 학습하기 위한 최종 Loss function이다.

모델이 training data의 **likelihood(AI가 원본이미지(X0)를 만들어내는 확률)**를 최대화하는 방향으로 학습

- 모든 latent 변수를 적분해야 하기 때문에 직접 계산하기 매우 어려워 위 식의 점수를 잘 받으면 고품질의 이미지를 만들 것을 보장
- 즉 계산이 안되는 **NLL 대신 ELBO를 활용해 ELBO를 최대화 하도록 학습을 진행**

ELBO란? 완벽하게 특징을 반영하는 데이터 분포를 구하는 것은 불가능하므로 하한선을 정해서 근사값을 찾는 것

빨간 박스 → **p(reverse process)와 q(forward process) 과정을 하나로 묶어주는 역할**

- NLL을 통해 q를 바탕으로 p가 잘 학습되고 있는지 비교해 그 차이를 환산 (p → 예측 분포 | q → 정답 분포)
- 이때 p와 q의 거리는 **KL Divergence (클백-라이블러 발산)** 수식을 통해 계산
Gradient descent을 활용해 KL 분산 값을 최소화하고 파라미터를 업데이트해 더 정확한 예측을 유도함

KL Divergence이란?

학습 과정에서 **p와 q 두 분포 간의 차이를 최소화**하기 위한 Loss Function의 핵심 요소로 사용 = 분포 간 차이가 0에 가까워질수록 p가 q를 완벽하게 재현하고 있다는 뜻

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

기존 방식은 x_t 를 만들기 위해 노이즈를 t번 섞어야했지만 Diffusion은 원본(x_0)에서 t단계 x_t 로 한번에 건너 뛰어 학습 속도를 절약하였다. = 임의의 시간 t에서 샘플링 X_t 가 가능하다는 것

$$\mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

한번에 샘플링이 가능하므로 효율적인 학습 유도하고 Loss function을 이와 같이 쓰면 분산 감소로 추가 개선한다. KL divergence로 $p_{\theta}(x_{t-1}|x_t)$ 를 비교하며 closed form 방식으로 계산할 수 있어 L을 쉽게 구할 수 있다.

Diffusion models and denosing autoencoder

$$\mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right],$$

L_T 는 VAE의 loss에서 regularization term과 상응하고, L_{t-1} 은 denoising process, L_0 은 VAE의 loss에서 reconstruction term과 상응한다.

(1) Forward process and LT

논문에서 $p(x_T) = N(x_T; 0, I)$ 으로 고정되어있고,
 $q(x_T|x_0) = N(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$ 으로 learnable parameter가 없기 때문에
 학습중에는 상수로 고정된다.

(2) Reverse process and L1:T-1

$$L_{t-1} - C = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon), \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \sqrt{1 - \alpha_t} \epsilon) \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (9)$$

$$= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (10)$$

- 이미지 전체를 한번에 생성하는 것이 아닌 **노이즈 값(ϵ)을 찾는 방식으로 모델을 학습하는 것**
- $\epsilon_\theta(\mathbf{x}_t, t) \rightarrow$ **특정 시점 t와 이미지 $\mathbf{x}_t$ 가 주어졌을때 해당 시점의 노이즈(ϵ)를 예측하는 네트워크**
- 수식을 이용해 평균 예측 문제를 노이즈 예측 문제로 바꿀 수 있어 loss가 훨씬 간단해짐

특히 저자들은 ϵ 을 예측하도록 loss term을 simplification하는게 성능이 좋다고 말함

(3) Algorithms

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t) \right\|^2$ 
6: until converged
  
```

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
  
```

Algorithm 1 : Training

- 데이터셋에서 원본($\mathbf{x}_0$)를 가져와 노이즈 섞을 정도를 랜덤하게 선택
- noise를 더해나가는 과정, $\text{network}(\epsilon_\theta, p_\theta)$ 가 t step에서 noise(ϵ)가 얼마만큼 더해졌는지를 학습
- 학습 시에는 특정 step 이미지가 가우시안 노이즈에서 얼마나 denoising되어있는지 예측하도록 학습됨

- Closed form으로 원본(x_0)만 가지고 효율적으로 학습 가능

Algorithm 2 : Sampling

- network를 학습한 후 가우시안 노이즈에서 시작해 순차적으로 denoising 하는 과정
- 학습 알고리즘과 반대과정으로 이미지 복구
- 매 단계마다 미세한 랜덤 노이즈 (z)를 더해 더 풍부한 이미지를 생성하도록 함
- 노이즈만 가지고 실행해 마르코프 체인 과정으로 진행

(4) Data Scaling

$$p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1) = \prod_{i=1}^D \int_{\delta_{-}(x_0^i)}^{\delta_{+}(x_0^i)} \mathcal{N}(x; \mu_{\theta}^i(\mathbf{x}_1, 1), \sigma_1^2) dx$$

- $D \rightarrow$ 데이터 차원
- $i \rightarrow$ 이미지 데이터의 픽셀
- 데이터 스케일링
 - $\{0, 1, \dots, 255\}$ 범위의 이미지 데이터를 $[-1, 1]$ 로 모델이 계산하기 편한 상태(선형적)로 스케일링모델이 생성한 이미지를 정수 픽셀로 변환해 저장하는 최종 출력 필터

(5) Simplified training objective

$$L_{simple}(\theta) := \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2]$$

- 저자들은 L_{t-1} loss term을 위와 같이 **simplification** 함
- 특정 시점의 이미지에 추가된 노이즈를 잘 예측하는 것을 목표로 학습함
- 복잡한 가중치를 제거하고 예측 오차만 계산
 - 기존 ELBO식 앞에 복잡한 가중치를 생략(가중치항은 t 에 대한 함수로 큰 t 에도 학습이 잘 되도록 가중치항 제거)
- t 가 작을때(노이즈가 거의 없는 상태) $\rightarrow$ 학습 강도 낮음 t 가 클때(노이즈가 많은 상태) $\rightarrow$ 학습 강도 높음

⇒ t 가 이미지 형태를 잘 다루게 되어 전체적인 이미지 구도가 더 선명하게 잡힘

Experiments

- 모든 실험에서 $T=1000$
- β_t 는 $\beta_1=10^{-4}$ 에서 $\beta_T=0.02$ 로 선형적으로 증가
- x_T 에서 signal-to-noise-ratio는 최대한 작게
 $(L_T = D_{KL}(q(x_T|x_0)|N(0, I)) \approx 10^{-5})$
- 신경망은 group normalization을 사용하는 U-Net backbone
(unmasked PixelCNN++과 비슷한 구조)
- Transformer sinusoidal position embedding으로 모델에게 시간 t 를 입력
- 16×16 feature map에서 self-attention 사용

Table 1: CIFAR10 results. NLL measured in bits/dim.

Model	IS	FID	NLL Test (Train)
Conditional			
EBM [11]	8.30	37.9	
JEM [17]	8.76	38.4	
BigGAN [3]	9.22	14.73	
StyleGAN2 + ADA (v1) [29]	10.06	2.67	
Unconditional			
Diffusion (original) [53]			≤ 5.40
Gated PixelCNN [59]	4.60	65.93	3.03 (2.90)
Sparse Transformer [7]			2.80
PixelIQN [43]	5.29	49.46	
EBM [11]	6.78	38.2	
NCSNv2 [56]		31.75	
NCSN [55]	8.87 ± 0.12	25.32	
SNGAN [39]	8.22 ± 0.05	21.7	
SNGAN-DDLS [4]	9.09 ± 0.10	15.42	
StyleGAN2 + ADA (v1) [29]	9.74 ± 0.05	3.26	
Ours (L , fixed isotropic Σ)	7.67 ± 0.13	13.51	≤ 3.70 (3.69)
Ours (L_{simple})	9.46 ± 0.11	3.17	≤ 3.75 (3.72)

Table 2: Unconditional CIFAR10 reverse process parameterization and training objective ablation. Blank entries were unstable to train and generated poor samples with out-of-range scores.

Objective	IS	FID
$\tilde{\mu}$ prediction (baseline)		
L , learned diagonal Σ	7.28 ± 0.10	23.69
L , fixed isotropic Σ	8.06 ± 0.09	13.22
$\ \tilde{\mu} - \tilde{\mu}_\theta\ ^2$	–	–
ϵ prediction (ours)		
L , learned diagonal Σ	–	–
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \tilde{\epsilon} - \epsilon_\theta\ ^2 (L_{\text{simple}})$	9.46 ± 0.11	3.17

표 2의 결과를 보면 논문에서 제안한 ϵ 예측은 고정 분산으로 variational bound에 대해 훈련할 때 μ 를 예측하는 것과 거의 비슷한 성능을 보이지만, 단순화된 목표로 훈련할 때 훨씬 더 나은 성능을 보인다.